FlowSTLC: Securing Information Flow with Types

CS315 Computer Security (2025 Fall)

Zhige Chen, Junqi Huang, Zhiyuan Cao

Department of Computer Science and Engineering, SUSTech

December 22, 2025

Some Notes

Since most of our audiences don't have a background in PL/Type Theory, we will minimize the formal mathematical context here and instead use an intuitive explanation.
For more formal specification/proofs, please refer to the project report.

The Problem

- Software mixes high-security data (passwords, private keys, etc.) with low-security data. This may lead to data leaks.
- A develop might accidentally print a password to a debug log.
- An attacker might try to infer secrets by observing public behavior.

The goal:

- Ensure *confidentiality*.
- *Noninterference*: changing a secret input should have *no observable effect* on public output.

What is Informational Flow Control?

Instead of just checking access (Can we read this file?), we check *flow* (Where does this data go?).
Static analysis: we want to catch these leaks at compile time.
We label data as either:

- `Pub` (Public/Low Security)
- `Sec` (Secret/High Security)

Core Idea: The "Security Box"

We use a concept called *graded modality*. Fundamentally, a modality is a box with a security label:

- We can seal both public and secret data into a secret-labelled box, but we can only open it under a high-security (secret) context.
- We can only seal public data into a public-labelled box, but we can open it regardless of the context.

How The Type System Works?

We use a *semiring* (A mathematical structure) to decide how security labels interact with each other:

- Order: $\texttt{Pub} \leq \texttt{Sec}$
- Addition as meet: $r + s = r \sqcap s$
- Multiplication as join: $r \cdot s = r \sqcup s$
- $0 = \texttt{Sec}$, $1 = \texttt{Pub}$

A small example:

```
1 if secret_passwd == "1234"
2     then true
3     else false
4
```

Implementation

How we built it:

- Language: Java 17.
- Lexer & Parser: Generated by ANTLR4.
- Type Checker: The core of the interpreter. Based on the bidirectional typing system to enforce the security rules.
- We also built an language server to provide basic semantic highlighting and hover hints.

Limitations & Future Work

Current status:

- We verify the system's capability through both formal proofs and implementation.
- **Limitations**:
  - It's a small academic language and lacks real-world programming constructs (no complex data structures, polymorphism, etc.)
  - The effectiveness of the system has not been verified at larger scale.

Future work:

- Extend the type system with more advanced features, e.g., *parametric polymorphism*, *subtyping*, *dependent typing*, other form of *effect/coeffect analysis*.
- Integrate our system into industrial languages/real-world code bases to evaluate its effectiveness.

Thanks!